Seminario de C++
Parte I

Departamento de Lenguajes y Sistemas Informáticos
Universidad de Alicante
Curso 2026-2027



Esta obra se publica con una licencia BY-NC-SA Creative Commons License

Index

1. Tipos abstractos de datos. Clases y objetos
2. Compilación y herramienta make
3. Operaciones canónicas
4. Sobrecarga de operadores
5. Librería STL
6. Depuración con Valgrind

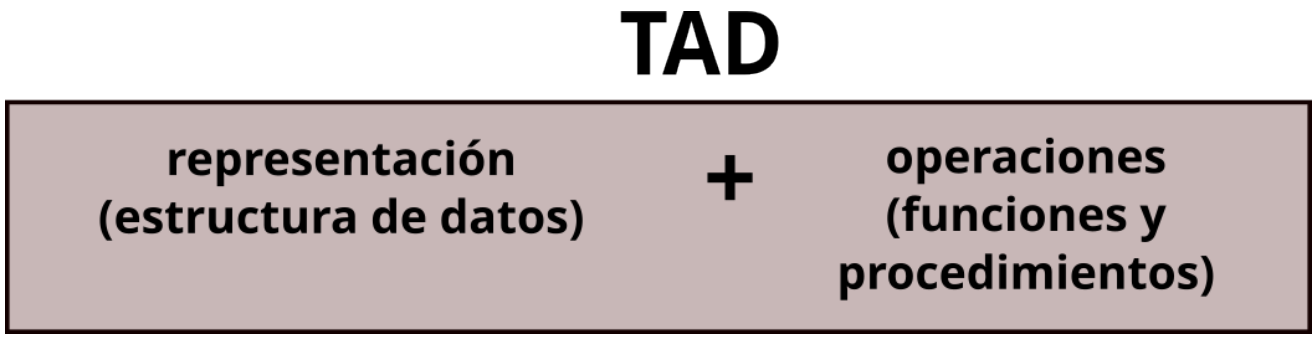
Index

1. Tipos abstractos de datos. Clases y objetos
2. Compilación y herramienta make
3. Operaciones canónicas
4. Sobrecarga de operadores
5. Librería STL
6. Depuración con Valgrind

Tipos Abstractos de Datos

- Los lenguajes de programación como Java o C++ permiten ampliar el lenguaje añadiendo sus propios tipos de datos.

- Un tipo de datos definido por el programador se denomina **tipo abstracto de datos (TAD)**, para distinguirlo de los datos predefinidos.



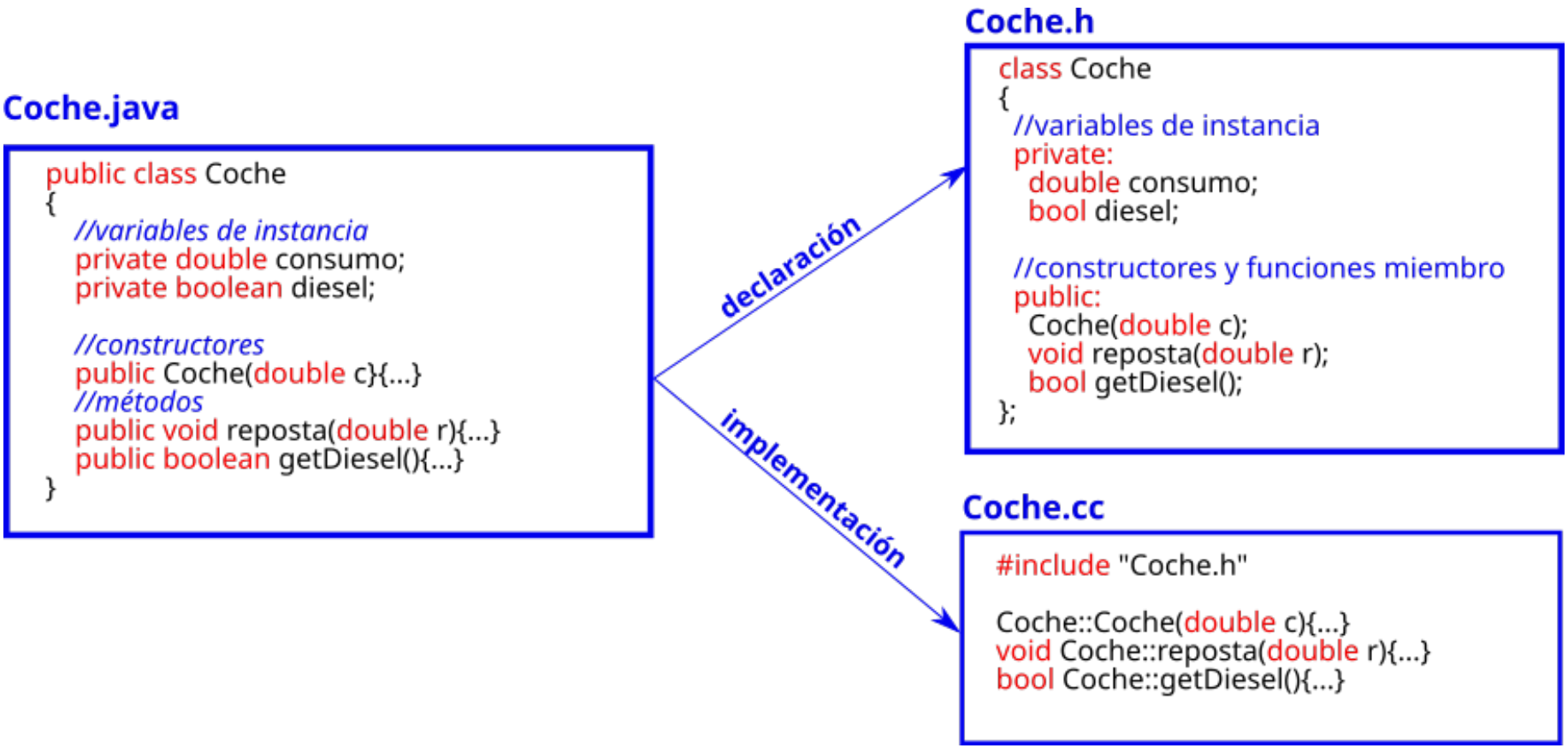
Clases y objetos en Java y C++

Las clases representan tipos de datos definidos por el usuario



Objeto: es una instancia de una clase determinada.

Declaración de una clase en Java y en C++



Declaración de una clase en C++ (I)

- los métodos se denominan *funciones miembro*, y normalmente sólo se escribe el prototipo en la declaración;

- las declaraciones se sitúan en el archivo `NombreClase.h` y la implementación de las funciones miembro en el archivo `NombreClase.cc`;

- Un **namespace** es un ámbito seguro para identificadores (evita colisiones de nombres). Un ejemplo típico de tales componentes es la **librería estándar de C++**. Tenemos varias opciones:

  - la sentencia `"using namespace std;"` permite escribir
    `cout << 3 << endl;`

  - sin dicha sentencia escribiríamos
    `std::cout << 3 << std::endl;`

Declaración de una clase en C++ (II)

- **Guardas de inclusión.** Permiten evitar la doble inclusión, evitando errores de compilación. Se añaden en el fichero de definición de la clase (.h)

```
#ifndef NombreClase_H //directiva de preprocesamiento
#define NombreClase_H
 ... declaracion de la clase ...
#endif
```

- **Funciones amigas.** Rompen la "ocultación de información" cuando:
  - no hay otra solución (sobrecarga de algún operador)
  - cuestiones de claridad
  - cuestiones de rendimiento del programa

  Declaración de una función *funcionA* amiga de una clase B. Dentro de la clase B:

```
friend tipo funcionA(parametros);
```

Declaración de una clase en C++ (III)

En la inclusión de librerías en un fichero (.h) o (.cc), se debe respetar un orden:

1. librerías del sistema, con el nombre entre < y >
2. librerías propias, con el nombre entre " "

Ejemplo:

```
#include <iostream>
#include <string>
#include <vector>
#include "Pixel.h"
```

Ejemplo de declaración de una clase en C++

```
//archivo Pixel.h
#ifndef PIXEL_H //directiva de preprocesamiento
#define PIXEL_H

#include <iostream>
#include <string>
using namespace std;

class Pixel{
  //funcion que puede acceder a las variables de Pixel
  friend double distancia( Pixel a, Pixel b );

  private:
    int x, y; //coordenadas del pixel
  public:
    Pixel(){x=0; y=0;} //constructor por defecto
    Pixel(int i, int j); //constructor parametrizado
    int getX(); //devuelve el valor de x
    void setX(int vx); //fija el valor de x
};
#endif
```

Ejemplo de implementación de una clase en C++

```
//archivo Pixel.cc
#include <cmath>
#include "Pixel.h"

Pixel::Pixel(int i, int j){ x = i; y = j; }

int Pixel::getX(){
  return x;
}

void Pixel::setX( int px ){
  x = px;
}

double distancia( Pixel a, Pixel b ){ //funcion amiga
  double v;
  v = pow((a.x - b.x), 2) + pow((a.y - b.y), 2);

  return sqrt(v);
}
```

En un programa principal.cc

```
#include "Pixel.h" //ya incluye iostream y namespace std

int main(int argc, char *argv[]){
    int i = 0;
    Pixel p1; //se invoca al constructor por defecto
    Pixel p2(i,2); //se invoca al constructor con 2 enteros por
        parametro

    p1.x = 3; //ERROR DE COMPILACION (x es privada)
    p1.setX(3); //OK, cambiamos el valor de la coordenada x
    cout << "valor de x = " << p1.getX() << endl;

    Pixel p3(2,5);
    cout << "distancia entre p2 y p3 = " << distancia(p2, p3) <<
        endl;
}
```

Index

1. Tipos abstractos de datos. Clases y objetos
2. Compilación y herramienta make
3. Operaciones canónicas
4. Sobrecarga de operadores
5. Librería STL
6. Depuración con Valgrind

Compilación y enlace

En el ejemplo anterior tenemos que compilar 2 ficheros .cc y enlazar los ficheros objeto .o para obtener el ejecutable.

```
g++ -g -c Pixel.cc  //compilacion -> genera Pixel.o
g++ -g -c principal.cc  //compilacion -> genera principal.o
g++ -g -o principal Pixel.o principal.o //enlace -> genera
    principal
```

Recomendable: activar los warning con la opción de compilación `-Wall`

Como se puede ver en este ejemplo, tenemos que ejecutar 3 órdenes para obtener el ejecutable `principal`.

Podemos utilizar la herramienta `make` sobre un fichero que ha de seguir una estructura determinada:

```
Objetivo: Dependencia/s
(tabulador) Orden a ejecutar
```

Compilación y enlace: herramienta make

Un ejemplo de fichero para los ficheros anteriores sería el siguiente:

```
principal: principal.o Pixel.o
(TAB) g++ -g -o principal principal.o Pixel.o

principal.o: principal.cc Pixel.h
(TAB) g++ -g -Wall -c principal.cc

Pixel.o: Pixel.h Pixel.cc
(TAB) g++ -g -Wall -c Pixel.cc
```

- si el fichero se llama `Makefile` es suficiente ejecutar `make`
- si el fichero se llama `makeprac` hay que ejecutar `make -f makeprac`
- las dependencias permiten detectar si hay que recompilar algún fichero o no

Index

1. Tipos abstractos de datos. Clases y objetos
2. Compilación y herramienta make
3. Operaciones canónicas
4. Sobrecarga de operadores
5. Librería STL
6. Depuración con Valgrind

Operaciones canónicas de una clase

- constructores (incluído constructor de copia)
- destructor
- operador =
- operador salida <<

```
//definicion de todas las operaciones canonicas para una clase
class Dato{
  friend ostream& operator<< (ostream &o, const Dato &p);
  private:
    int dato;
    string cadena;
    static int cantidad;
  public:
    Dato(); //constructor por defecto
    Dato(int a, string b); //constructor con parametros
    Dato(const Dato &p); //constructor de copia
    ~Dato(); //destructor
    Dato & operator = (const Dato &p); //operador asignacion
};
```

Operaciones canónicas de una clase: constructores

Se autoinvocan en diferentes circunstancias, que pueden ser en la declaración de objetos, asignación, salida de ámbito o tipo de paso de parámetro.

```
int Dato::cantidad = 0; //inicializacion de variable estatica

//constructor por defecto
Dato::Dato(){
  dato = 0;
  cadena = "";
  cantidad++;
}
//constructor con paso de parametro
Dato::Dato( int d, string c ){
  dato = d;
  cadena = c;
  cantidad++;
}

//declaracion de objetos de tipo Dato
Dato d1(3, "casa"); //objeto con memoria estatica
```

Operaciones canónicas de una clase: constructor de copia

```
//se invoca automaticamente en un paso (o devolucion) por valor
Dato::Dato( const Dato &p ){
  dato = p.dato;
  cadena = p.cadena;
}

//declaracion de objetos de tipo Dato
Dato d1(3, "casa"); //objeto con memoria estatica
Dato d3(d1); //d3 es una copia de d1 - se invoca al constructor
    de copia
```

Operaciones canónicas: destructor

- Se encarga de de liberar todos los recursos que el objeto mantenga en uso, como por ejemplo la memoria dinámica reservada o los ficheros abiertos.
- Se invoca automáticamente cuando sale de ámbito un objeto o cuando se utiliza la función **delete** con un objeto.

```
Dato::~Dato(){
  // Aqui se liberaria la memoria dinamica reservada
  // y otros recursos como ficheros abiertos
}
```

- Si el objeto no tiene recursos que haya que liberar, no es necesario definirlo.
- Si no se define, el compilador genera uno por defecto que no hace nada.

Operaciones canónicas: sobrecarga operador asignación

- Se usa para poder asignar un objeto a otro como si fueran variables de tipo primitivo, de manera que se crea una copia del objeto.
- El compilador genera uno que asigna atributo a atributo, pero hay que redefinirlo si la clase tiene variables de memoria dinámica.

```
Dato & Dato::operator =( const Dato &obj ) {
  if( this!=&obj ){
    dato = obj.dato;
    cadena = obj.cadena;
  }
  return *this;
}

//uso
Dato d1(3, "casa");
Dato d2 = d1; //se invoca al operador =
```

Operaciones canónicas: sobrecarga operador salida

- Se usa para poder mostrar el contenido del objeto utilizando el operador $<<$.
- Debido a que es un objeto de otra clase (de tipo fichero) que necesita acceder a la parte privada del objeto, éste operador se sobrecarga a través de una **función amiga**, ya que las funciones amigas pueden acceder a la parte privada de los objetos.

```
//declaracion
friend ostream & operator<<( ostream &out, const Dato &obj );

//implementacion
ostream& operator <<( ostream &out , const Dato &obj ) {
  out << obj.dato << " -> " << obj.cadena << endl ;
  return out;
}

//uso
Dato d1(3, "casa");
cout << d1 << endl; // 3 -> casa
```

Paso de parámetros en C++: valor y referencia

- Paso por valor: se invoca el constructor de copia del objeto pasado por parámetro
- Paso por referencia: sólo se toma la dirección de memoria de la variable, por lo que no se realiza la copia del objeto (se debe anteponer el símbolo &)

```
//se invoca el constructor de copia de Dato
void pasoValor( Dato obj ) { ... }

//accedemos a obj sin hacer copia, pero lo podemos modificar
void pasoReferencia( Dato &obj ) { ... }

//accedemos a obj sin hacer copia y no podemos modificarlo dentro
     de la funcion (daria un error de compilacion intentarlo)
void pasoReferencia( const Dato &obj ) { ... }

//devolvemos por valor
Dato devValor( ... ){ ... }
```

Index

1. Tipos abstractos de datos. Clases y objetos
2. Compilación y herramienta make
3. Operaciones canónicas
4. Sobrecarga de operadores
5. Librería STL
6. Depuración con Valgrind

Sobrecarga de operadores en C++

- La sobrecarga de operadores es un mecanismo que permite ampliar la capacidad del lenguaje de programación C++.
- La sobrecarga se puede hacer dentro de la clase (función miembro), y como funciones amigas de la clase.
- Las típicas sobrecargas que se suelen utilizar son:

+ (suma)	- (resta)
* (multiplicación)	/ (división)
= (asignación)	++ (incremento)
== (comparación)	< (orden)
<< (salida)	>> (entrada)

Ejemplo de una clase

```
#include <iostream>
using namespace std;

class Par{
    // operadores no miembros
    friend ostream& operator << (ostream &o,const Par &p);

  private:
    int i, j;
  public:
    // constructores
     ...
    // operadores miembros
    Par operator + (const Par &p);
    Par& operator = (const Par &p);
    Par& operator ++(); //pre-incremento
    bool operator ==(const Par &p) const;
};
```

Implementación de los operadores

```
Par Par::operator + (const Par &p)
{
    Par c;
    c.i = i + p.i;
    c.j = j + p.j;
    return c;
}

Par& Par::operator++()
{
    i++;
    j++;
    return *this;
}

Par& Par::operator = (const Par &p)
{
    if( this != &p ){ //Comprueba que no se este intentando
        igualar un objeto a si mismo
        i = p.i;
        j = p.j;
    }
    return *this;
}
```

Operadores

```
bool Par::operator == (const Par &p) const
{
    return i == p.i && j == p.j;
}

// implementacion de operadores no miembros (friend)
ostream & operator << (ostream &o,const Par &p)
{
    o << "(" << p.i << ", " << p.j << ")";
    return o;
}
```

En un programa principal.cc

```
int main ( int argc , char * argv []) {
  Par p1; // se invoca al constructor por defecto (0,0)
  Par p2(1, 2);

  Par p3 = p1 + p2; //operador suma
  cout << "p3 (p1+p2) = " << p3 << endl;

  ++p3; //operador pre-incremento
  cout << "p3 (tras ++)= " << p3 << endl;

  p1 = p2; //operador asignacion
  cout << "p1 (copia de p2) = " << p1 << endl;


  //operador comparacion
  if( p1 == p2 ) cout << "p1 y p2 son iguales" << endl;
  else cout << "p1 y p2 son diferentes" << endl;

  if( p1 == p3 ) cout << "p1 y p3 son iguales" << endl;
  else cout << "p1 y p3 son diferentes" << endl;

}
```

Index

1. Tipos abstractos de datos. Clases y objetos
2. Compilación y herramienta make
3. Operaciones canónicas
4. Sobrecarga de operadores
5. Librería STL
6. Depuración con Valgrind

Librería STL

- La STL (Standard Template Library – Biblioteca Estándar de Plantillas) es una biblioteca de C++ que básicamente está compuesta de:
  - Contenedores: permiten almacenar objetos;
  - Algoritmos de uso frecuente;
  - Iteradores para recorrer los elementos incluídos en los contenedores;
- Tipos de contenedores: vectores, pilas, listas, colas, árboles ...

El contenedor **vector**

- Permite almacenar cero o más elementos del mismo tipo, pudiendo acceder a ellos individualmente mediante un índice (acceso aleatorio).
- El número de elementos del vector puede variar dinámicamente.
- La gestión de memoria se hace de manera transparente al usuario.
- Puede albergar elementos de cualquier tipo.
- Ejemplo de uso:

```
#include <vector>     //libreria del contenedor

vector<int> v1; //vector de enteros

vector<Pixel> v2; //vector de Pixel, definido por el usuario

vector<string> v3; //vector de string
```

Algunas funcionalidades del **vector**

Supongamos que tenemos el siguiente vector:

```
vector<int> v1;
```

- obtener el número de elementos almacenados

```
v1.size();
```

- almacena un elemento al final del vector

```
v1.push_back(3);
```

- acceso aleatorio a los elementos de un vector

```
v1[0] = 7; //sin comprobacion de rango
```

Tratamiento de los índices del **vector**

- Para cualquier índice n, lo siguiente debe ser verdadero:

```
0 <= n < v1.size()
```

- Ejemplos:

```
vector<int> v2;

v2[0] = 25; //esto da error, porque el vector esta vacio

v2.push_back(15);
v2[0] = 25; //correcto
```

Operaciones del tipo vector

- `size_t size();` //devuelve el número de elementos almacenados en el vector. (size_type es un entero sin signo).
- `bool empty();` //devuelve true si el número de elementos es cero y false en caso contrario.
- Funciones para insertar:
  - `void push_back(const T& x);` //inserta un elemento `x` al final del vector. Aumenta el tamaño del vector en 1. T es el tipo de datos que almacena el vector.
  - `iterator insert( iterator loc, const T& val );` //Inserta `val` antes de `loc` [1].

[1] loc representa una posición en el vector

Operaciones del tipo vector: borrados y operadores

- `iterator erase (iterator pos);` // borra el elemento que está en la posición indicada por `pos`.
- `void pop_back();` //elimina el último elemento. Reduce el tamaño en 1.
- `void clear();` //borra todos los elementos de un vector.
- Operadores:
  - Operador asignación (=) sustituye el contenido de un vector por una copia del contenido del otro vector.
  - Operador comparación (==) comprueba si dos vectores contienen elementos con el mismo contenido.
  - Operador subíndice ([ ]) devuelve una referencia a un elemento del vector. El rango va de 0 a size()-1.

Ejemplo de uso de **vector**

```
#include <iostream>
#include <vector>
using namespace std;

int main(int argc, char * argv [])
{
    vector<int> v1(2, 1); //vector de 2 elementos con valor 1
    v1.push_back(3); v1.push_back(5);

    for(size_t i=0; i<v1.size(); i++)
      cout << v1[i] << " ";
    cout << endl;

    v1.insert( v1.begin()+1, 7 ); //inserto en pos 1
    for(size_t i=0; i<v1.size(); i++)
      cout << v1[i] << " ";
    cout << endl;

    v1.erase(v1.begin()+2); //borro pos 2
    for(size_t i=0; i<v1.size(); i++)
      cout << v1[i] << " ";
    cout << endl;
}
```

Compilar un único fichero: `g++ -o vector vector.cc`

Index

1. Tipos abstractos de datos. Clases y objetos
2. Compilación y herramienta make
3. Operaciones canónicas
4. Sobrecarga de operadores
5. Librería STL
6. Depuración con Valgrind

Herramienta para depuración: valgrind

Sirve para detectar errores comunes de manejo de memoria con la herramienta Memcheck, como:

- acceso indebido a memoria (índices incorrectos en vectores)
- uso de variables no inicializadas
- liberación incorrecta de la memoria

Para poder utilizar esta herramienta tenemos que compilar los programas con la opción de generación de información para depuración (-g).

```
valgrind [opciones] ejecutable-de-nuestro-programa
```

Lo compilamos con la opción de depuración y lo ejecutamos con valgrind:

```
g++ -g -o ejemplo ejemplo.cc
valgrind ./ejemplo
```

Se genera una salida que nos informa del problema encontrado, si lo hay.

Ejemplo de uso de valgrind (1)

Aparte de la salida estándar, siempre nos encontraremos este resumen al final. En este caso no ha habido ningún problema con la gestión de memoria aunque lo parezca. Se ha enmarcado la línea en la que hay que fijarse realmente:

```
==813505== Memcheck, a memory error detector
==813505== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==813505== Using Valgrind-3.15.0 and LibVEX; rerun with -h for copyright info
==813505== Command: vectorSTL
==813505==
1 2
==813505==
==813505== HEAP SUMMARY:
==813505==     in use at exit: 0 bytes in 0 blocks
==813505==   total heap usage: 4 allocs, 4 frees, 73,740 bytes allocated
==813505==
==813505== All heap blocks were freed -- no leaks are possible
==813505==
==813505== For lists of detected and suppressed errors, rerun with: -s
==813505== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Ejemplo de uso de valgrind (2)

En este caso hemos accedido a una zona de memoria no reservada, en concreto hemos intentado borrar un elemento del vector en un índice que no existe.

Entre toda la información de salida que aparece al ejecutar valgrind nos encontramos una referencia a la línea de código donde se ha producido el error.

```
==815155==    by 0x109814: std::vector<int, std::allocator<int> >::erase(__gnu_cxx::
st*, std::vector<int, std::allocator<int> > >) (stl_vector.h:1428)
==815155==    by 0x109429: main (vectorSTL.cc:17)
==815155==  If you believe this happened as a result of a stack
==815155==  overflow in your program's main thread (unlikely but
==815155==  possible), you can try to increase the size of the
==815155==  main thread stack using the --main-stacksize= flag.
==815155==  The main thread stack size used in this run was 8388608.
==815155==
==815155== HEAP SUMMARY:
==815155==     in use at exit: 8 bytes in 1 blocks
==815155==   total heap usage: 4 allocs, 3 frees, 73,740 bytes allocated
==815155==
==815155== LEAK SUMMARY:
==815155==    definitely lost: 0 bytes in 0 blocks
==815155==    indirectly lost: 0 bytes in 0 blocks
==815155==      possibly lost: 0 bytes in 0 blocks
==815155==    still reachable: 8 bytes in 1 blocks
==815155==         suppressed: 0 bytes in 0 blocks
==815155== Rerun with --leak-check=full to see details of leaked memory
==815155==
==815155== For lists of detected and suppressed errors, rerun with: -s
==815155== ERROR SUMMARY: 6180535 errors from 3 contexts (suppressed: 0 from 0)
Violación de segmento (`core' generado)
```